

Project Milestone 1 - Report

Swann Ade, Antoine Heurtier and Elyes Ben Abid

CS-233 - Introduction to Machine Learning

1 Introduction

This report presents our implementation of three machine learning methods applied to the Gaming and Mental Health Dataset, a dataset composed of 2000 samples with 13 features. The goal is to predict either the addiction score (regression, 0-10) or the addiction level (classification, Low/Medium/High) from gaming habits.

We implement Linear Regression, Logistic Regression, and K-Nearest Neighbors from scratch using course formulas through NumPy, and evaluate them using MSE for regression and accuracy/F1-score for classification.

2 Data Preparation

The 1600 provided training samples were split 80/20 into 1280 training and 320 validation samples using a shuffled permutation with a random seed (fixed to 100 in the code). The validation set was used to tune hyperparameters without touching the test set, and this is done by passing the `--test` flag.

Feature normalization is then applied after the split using provided utility functions: means and standard deviations are computed on the training set only, then applied to both training and validation sets, as computing statistics before the split would "leak" validation information into the model. Adding a bias is not done in this step of the pipeline as not all methods (such as KNN) require a bias to be added.

3 Methods

3.1 Linear Regression

For Linear Regression, the weights are estimated using the closed-form normal equation : $\mathbf{W} = \mathbf{X}^+ \mathbf{y}$ where $\mathbf{X}^+$ denotes the "*Moore-Penrose*" pseudo-inverse.

A bias term was also prepended to the feature matrix, avoiding the constraint of passing through the origin. We also verify through an "assert" that the model was correctly trained before prediction.

This gives the exact least-squares solution in a single step, with no hyperparameters to tune and no convergence issues. Training and prediction are extremely fast as shown in the **Results** section, reflecting the efficiency of the closed-form solution over iterative methods.

3.2 Logistic Regression

For Logistic Regression, the weights are learned via gradient descent. Starting from $\mathbf{W}$ to $\mathbf{0}$, we iteratively update:

$$\mathbf{W}_{i+1} = \mathbf{W}_i - \eta \nabla_{\mathbf{W}} \mathcal{L}(\mathbf{W})$$

where $\mathcal{L}$ denotes the cross-entropy loss and η the learning rate. The learning rate η and the number of iterations constitute the hyperparameters of the model.

To select them, we performed a hyperparameter search and found that $\eta = 1.7$ with **iters** = 1000 yielded the best results. These are the values used throughout the results section.

3.3 K-Nearest Neighbors

The K-Nearest Neighbors (KNN) algorithm was implemented for both classification and regression tasks. Unlike the other two methods, KNN is a non-parametric model with no explicit training phase; the `fit` method simply stores the training data and labels.

To optimize performance, we used a fully vectorized NumPy implementation to compute the Euclidean distances between query points and training samples:

$$d(\mathbf{x}, \mathbf{x}') = \sqrt{\|\mathbf{x}\|^2 + \|\mathbf{x}'\|^2 - 2\mathbf{x}\mathbf{x}'^T}$$

For a given query point, the K closest neighbors are selected. In classification, the prediction is the majority vote among these neighbors, while for regression, it is the mean of their target values. We conducted a grid search on the hyperparameter $K \in \{1, 5, 11, 21\}$ and found that $K = 11$ provided the best generalization on the validation set for both tasks, effectively balancing the bias-variance tradeoff.

4 Results

Method	F1	MSE	Accuracy	Train time	Predict time
Linear Regression	-	0.994	-	2.4ms	0.02ms
Logistic Regression	0.740	-	86.6%	214ms	0.041ms
KNN (K=11, Reg)	-	1.970	-	184 ms	62.4 ms
KNN (K=11, Classif)	0.541	-	78.75%	217 ms	66.9 ms

Table 1: Test set results and runtime (*tested on an Apple Macbook Pro M3*).

5 Discussion & Conclusion

The experimental results highlight significant differences between the models in terms of performance and efficiency.

Performance: The parametric models (Linear and Logistic Regression) outperformed KNN on this specific dataset. Specifically, Linear Regression achieved a much lower MSE (0.994) compared to KNN (1.970), and Logistic Regression reached a higher accuracy (86.6% vs 78.75%). This suggests that the relationship between gaming habits and addiction levels follow a linear trend that parametric models manage to capture efficiently.

Hyperparameters and Visualization: Both Logistic Regression and KNN required careful tuning. We implemented visualizations to monitor the training loss and validation performance, which proved essential for selecting the learning rate for gradient descent. For KNN, we observed clear overfitting at $K = 1$ (100% training accuracy but lower test performance). Increasing K to 11 improved generalization, while higher values led to underfitting.

Experimental Explorations: To go beyond the basic requirements, we tested several enhancements. We implemented L2 regularization to prevent overfitting in Logistic Regression, but it provided no significant gain, likely because the dataset size is sufficient relative to the number of features. We also explored sample re-weighting to account for potential class imbalances. However, this did not improve the macro F1-score, suggesting that the features provide enough separation.

Efficiency: Linear Regression was the fastest method due to its one-step closed-form solution. In contrast, KNN’s prediction time is significantly higher (around 60ms) because it must compute distances against all 1280 training samples for every new prediction. While KNN is “lazy” and fast to train, its inference cost scales linearly with the dataset size, making it less suitable for large-scale applications.

In conclusion, every model has its strengths and weaknesses, but they all come to the conclusion that gaming habits negatively impact player’s mental health.